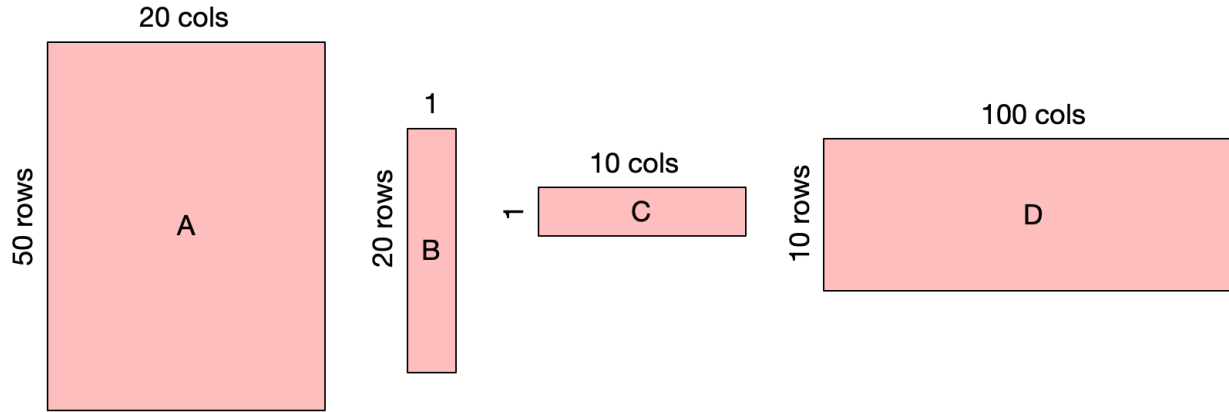


CS 3510 Algorithms: Chain Matrix Multiplication and Knapsack on Integers

Aaron Hillegass
Georgia Tech

Parenthesizations



There are 5 different parenthesizations:

$A(B(CD))$, $(AB)(CD)$, $A((BC)D)$, $((AB)C)D$, $(A(BC))D$

What is the fastest way?

How many parenthesizations?

For n terms, let C_{n-1} be the number of parenthesizations.

Example: $C_2 = 2$ because: $(AB)C$, $A(BC)$.

If we define $C_0 = 1$, then

$$C_n = \sum_{k=1}^n C_{k-1} C_{n-k}$$

Why?

Catalan Numbers

C_i are known as “The Catalan Numbers”:

1, 1, 2, 5, 14, 42, 132, 429, 1430, 4862, 16796, 58786, ...

Easy formulas:

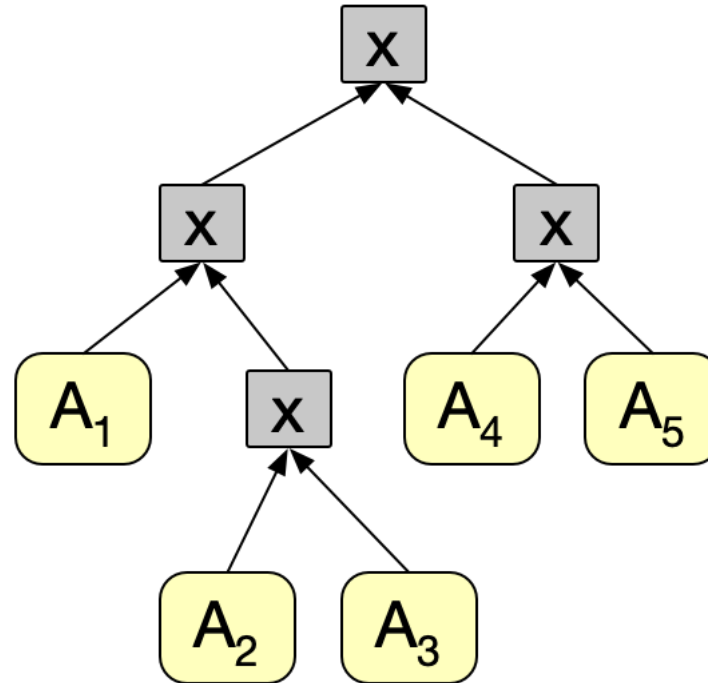
$$C_n = \frac{(2n)!}{(n+1)!n!} = \binom{2n}{n} - \binom{2n}{n+1}$$

How fast does it grow?

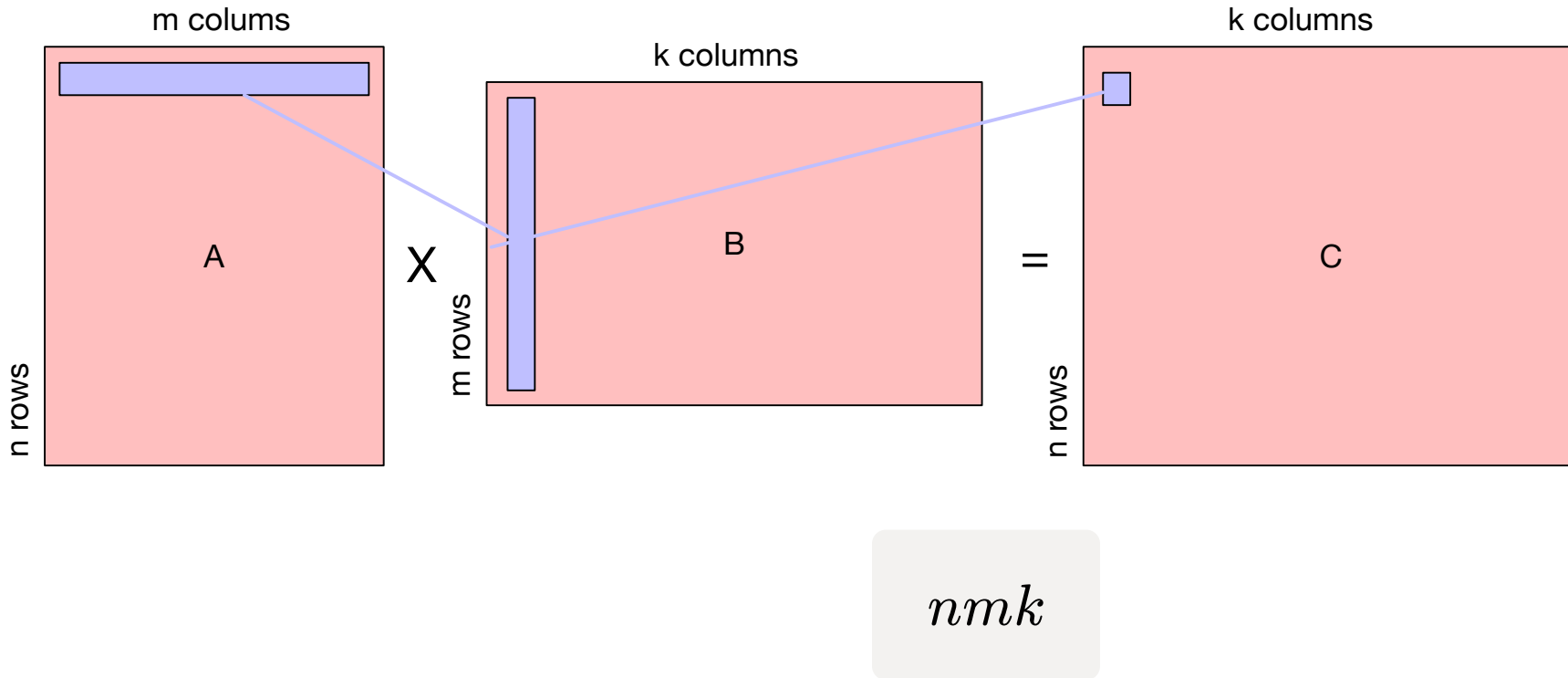
$$C_n \approx \frac{4^n}{n^{\frac{3}{2}}\sqrt{\pi}}$$

Parenthesization as tree

$(A_1(A_2A_3))(A_4A_5)$

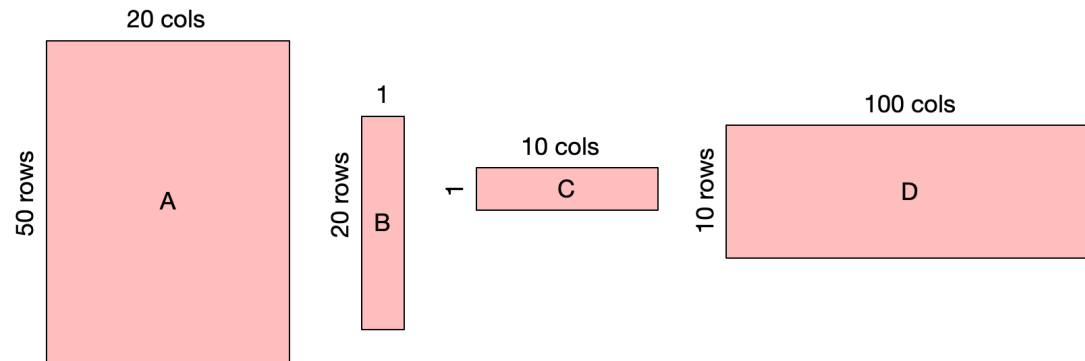


A good approximation of cost?



Pessimistic? So what?

Are some better than others?



$$A((BC)D) = (20 \times 1 \times 10) + (20 \times 10 \times 100) + (50 \times 20 \times 100) = 120,200 \text{ operations}$$

$$(AB)(CD) = (50 \times 20 \times 1) + (1 \times 10 \times 100) + (50 \times 1 \times 100) = 7,000 \text{ operations}$$

A sequence of n matrices $[A_1, A_2, \dots, A_n]$

A sequence of $n + 1$ dimensions $[d_0, d_1, \dots, d_n]$

Where A_i will be $d_{i-1} \times d_i$.

Recursion: Best split point?

- Cost of $(A_1)(A_2A_3A_4A_5) = d_0d_1d_5 + \text{cost}(A_2A_3A_4A_5)$
- Cost of $(A_1A_2)(A_3A_4A_5) = \text{cost}(A_1A_2) + d_0d_2d_5 + \text{cost}(A_3A_4A_5)$
- Cost of $(A_1A_2A_3)(A_4A_5) = \text{cost}(A_1A_2A_3) + d_0d_3d_5 + \text{cost}(A_4A_5)$
- Cost of $(A_1A_2A_3A_4)(A_5) = \text{cost}(A_1A_2A_3A_4) + d_0d_4d_5$

$\text{dp}[i, j] = \min \text{ of}$

- $\text{cost}(A_i)(A_{i+1} \dots A_j) = d_{i-1} d_i d_j + \text{cost}(A_{i+1} A_{i+2} \dots A_j)$
- $\text{cost}(A_i A_{i+1})(A_{i+2} \dots A_j) = \text{cost}(A_i A_{i+1}) + d_{i-1} d_{i+1} d_j + \text{cost}(A_{i+2} \dots A_j)$
- ...
- $\text{cost}(A_i \dots A_{j-1})(A_j) = \text{cost}(A_i \dots A_{j-1}) + d_{i-1} d_{j-1} d_j$

Dynamic Programming

Let dp be a $1..n \times 1..n$ array.

For $i \leq j$, $dp[i, j]$ is min cost of product $A_i \dots A_j$

Base case: $dp[i, i] := 0$

Solution: $dp[1, n]$

$[A_{9 \times 5}, A_{5 \times 3}, A_{3 \times 2}, A_{2 \times 10}, A_{10 \times 4}]$

		Last dimension of last matrix					
		1	2	3	4	5	
		5	3	2	10	4	
First dimension of first matrix	1	9	0	?	?	?	?
	2	5		0	?	?	?
	3	3			0	?	?
	4	2				0	?
	5	10					0

Recurrence: try all splits

For any $\text{dp}[i, j]$, try all $i \leq k < j$:

Split: $(A_i \dots A_k)(A_{k+1} \dots A_j)$

$$f(k) = \text{dp}[i, k] + d_{i-1}d_kd_j + \text{dp}[k+1, j]$$

$$\text{dp}[i, j] := \min_{i \leq k < j} f(k)$$

The no-choice cases

$$\text{dp}[0,2] = d_0 d_1 d_2 = (9)(5)(3) = 135$$

$$\text{dp}[1,3] = d_1 d_2 d_3 = (5)(3)(2) = 30$$

$$\text{dp}[2,4] = d_2 d_3 d_4 = (3)(2)(10) = 60$$

$$\text{dp}[3,5] = d_3 d_4 d_5 = (2)(10)(4) = 80$$

		1	2	3	4	5
		5	3	2	10	4
1	9	0	135	?	?	?
2	5		0	30	?	?
3	3			0	60	?
4	2				0	80
5	10					0

The 2 possibilities cases

	1	2	3	4	5	
	5	3	2	10	4	
	0	135	120	?	?	1
		0	30	130	?	2
			0	60	104	3
				0	80	4
					0	5

Cost of $(A_1A_2A_3) = dp[1,3] = \min$

- $k=1: A_1(A_2A_3): (9)(5)(2) + 30 = 120$
- $k=2: (A_1A_2)A_3: 135 + (9)(3)(2) = 189$

The 3 possibilities cases

	1	2	3	4	5	
	5	3	2	10	4	
	0	135	120	300	?	1
		0	30	130	150	2
			0	60	104	3
				0	80	4
					0	5

Cost of $(A_1A_2A_3A_4) = dp[0,4] = \min$

$k=1: A_1(A_2A_3A_4): (9)(5)(10) + 130 = 580$

$k=2: (A_1A_2)(A_3A_4): 135 + (9)(3)(10) + 60 = 465$

$k=3: (A_1A_2A_3)A_4: 120 + (9)(2)(10) = 300$

The final case: 4 possibilities

	1	2	3	4	5	
	5	3	2	10	4	
	0	135	120	300	272	1
		0	30	130	150	2
			0	60	104	3
				0	80	4
					0	5

Cost of $(A_1 A_2 A_3 A_4 A_5) = dp[0,5] = \min$

k=1: $A_1(A_2 A_3 A_4 A_5): (9)(5)(4) + 150 = 330$

k=2: $(A_1 A_2)(A_3 A_4 A_5): 135 + (9)(3)(4) + 104 = 347$

k=3: $(A_1 A_2 A_3)(A_4 A_5): 120 + (9)(2)(4) + 80 = 272$

k=4: $(A_1 A_2 A_3 A_4)A_5: 300 + (9)(10)(4) = 660$

But *what* is the best parenthization?

Backtracking

18/35

$$(A_{9 \times 5}(A_{5 \times 3}, A_{3 \times 2}))(A_{2 \times 10}, A_{10 \times 4})$$

$$(5 \times 3 \times 2) + (9 \times 5 \times 2) + (2 \times 10 \times 4) + (9 \times 2 \times 4)$$

$$30 + 90 + 80 + 72 = 272$$

		1	2	3	4	5
		5	3	2	10	4
1	9	0	135	120	300	272
2	5		0	30	130	150
3	3			0	60	104
4	2				0	80
5	10					0

The diagram illustrates the backtracking process for matrix multiplication. It shows a sequence of operations starting from the top-left cell (1,1) and moving towards the bottom-right cell (5,5). The values in the cells represent the cost of the operation at that step. The sequence of operations is indicated by arrows: (1,1) to (1,2), (1,2) to (1,3), (1,3) to (1,4), (1,4) to (1,5), (1,5) to (2,6), (2,6) to (3,5), (3,5) to (4,4), (4,4) to (5,3), and (5,3) to (5,4). The values in the cells are: (1,1)=9, (1,2)=0, (1,3)=135, (1,4)=120, (1,5)=300, (1,6)=272, (2,1)=5, (2,2)=0, (2,3)=30, (2,4)=130, (2,5)=150, (3,1)=3, (3,2)=0, (3,3)=60, (3,4)=104, (4,1)=2, (4,2)=0, (4,3)=80, (4,4)=0, (5,1)=10, (5,2)=0.

Just remember best k for each cell.

Backtracking

$$(A_1(A_2A_3))(A_4A_5)$$

		1	2	3	4	5
		5	3	2	10	4
1	9	0	135	120	300	272
2	5		0	30	130	150
3	3			0	60	104
4	2				0	80
5	10					0

		1	2	3	4	5
		5	3	2	10	4
1	9	0	1	1	3	3
2	5		0	2	3	3
3	3			0	3	3
4	2				0	4
5	10					0

Gathering data

```
1 define chain( $n, d : \text{array}[0..n]$ ):  
2    $\text{dp} := \text{array}[1..n, 1..n]$  of zeros  
3    $\text{best\_k} := \text{array}[1..n, 1..n]$   
4   for  $\text{seqlen} \in 2..n$  :  
5     for  $i \in 1..(n - \text{seqlen} + 1)$ :  
6        $j := i + \text{seqlen} - 1$   
7        $\text{bv} := \text{MAX}$   
8       for  $k \in [i..j - 1]$ :  
9          $v := \text{dp}[i, k] + d[i - 1]d[k]d[j] + \text{dp}[k + 1, j]$   
10        if  $v < \text{bv}$ :  
11           $\text{bv} = v; \text{bk} = k$   
12         $\text{dp}[i, j] := \text{bv}$   
13         $\text{best\_k}[i, j] := \text{bk}$ 
```

Showing results

```
print("Min ops: " + dp[1, n]) // Min ops: 272
print(split_string(best_k, 1, n) // ((A1)(A2.A3))(A4.A5)
```

```
1 define split_string(best_k, i, j):
2   if i == j:
3     return "A" + i
4   if i + 1 == j:
5     return "A" + i + ".A" + j
6   k = best_k[i, j]
7   return "(" + split_string(best_k, i, k) + ")" + "(" + split_string(best_k, k + 1, j)
```

Memory grows slower than the runtime:

- Memory: $O(n^2)$
- Runtime: $O(n^3)$

Your knapsack holds W kg.

There are $1 \dots n$ items with weight w_i and
valued v_i .

Which do you steal?

There are 2^n possible combinations.

Compute the value of each.

Take the argmax.

Greedy Algorithm

- Sort items by decreasing “value density”: $\frac{v_i}{w_i}$
- Start at highest value density and keep packing until full.
- When does this definitely get you the highest possible value?

Weight: 7kg
Value: \$700

Knapsack holds 9 kg

Weight: 3kg
Value: \$250

Weight: 3kg
Value: \$250

Weight: 3kg
Value: \$250

- **Fractional Knapsack:** The treasure is pellets/liquid so you can take fractions
- **0/1 Knapsack:** You take the whole item or none.

Dynamic Programming Solution?

Dynamic Programming Solution?

Works when weights are integers.

Let $dp[r, i]$ represent the max value for weight up to r , picking from items $1, \dots, i$.

i goes from 0 through n .

r goes from 0 through W .

Base cases:

- $dp[0, i] = 0$
- $dp[r, 0] = 0$

Solution: $dp[W, n]$

Recurrence?

		Last item added to options				
		0	1	2	3	4
			2,2.00	1,0.75	1,1.05	2,2.50
Weight in kg.	0	0	0	0	0	0
	1	0	?	?	?	?
	2	0	?	?	?	?
	3	0	?	?	?	?
	4	0	?	?	?	?
	$W = 5$	0	?	?	?	?

Recurrence?

- $w_i > r$? Can't steal item i :

$$\text{dp}[r, i] := \text{dp}[r, i - 1]$$

If $w_i \leq r$, you might steal it. If it is the most profitable move.

- Steal is best?

$$\text{dp}[r, i] := v[i] + \text{dp}[r - w[i], i - 1]$$

- No steal is best?

$$\text{dp}[r, i] := \text{dp}[r, i - 1]$$

Last item added to options

	0	1	2	3	4
		2,2.00	1,0.75	1,1.05	2,2.50
0	0	0	0	0	0
1	0	0	0.75	1.05	1.05
2	0	2.0	2.0	2.0	2.5
3	0	2.0	2.75	3.05	3.55
4	0	2.0	2.75	3.80	4.50
W = 5	0	2.0	2.75	3.80	5.55

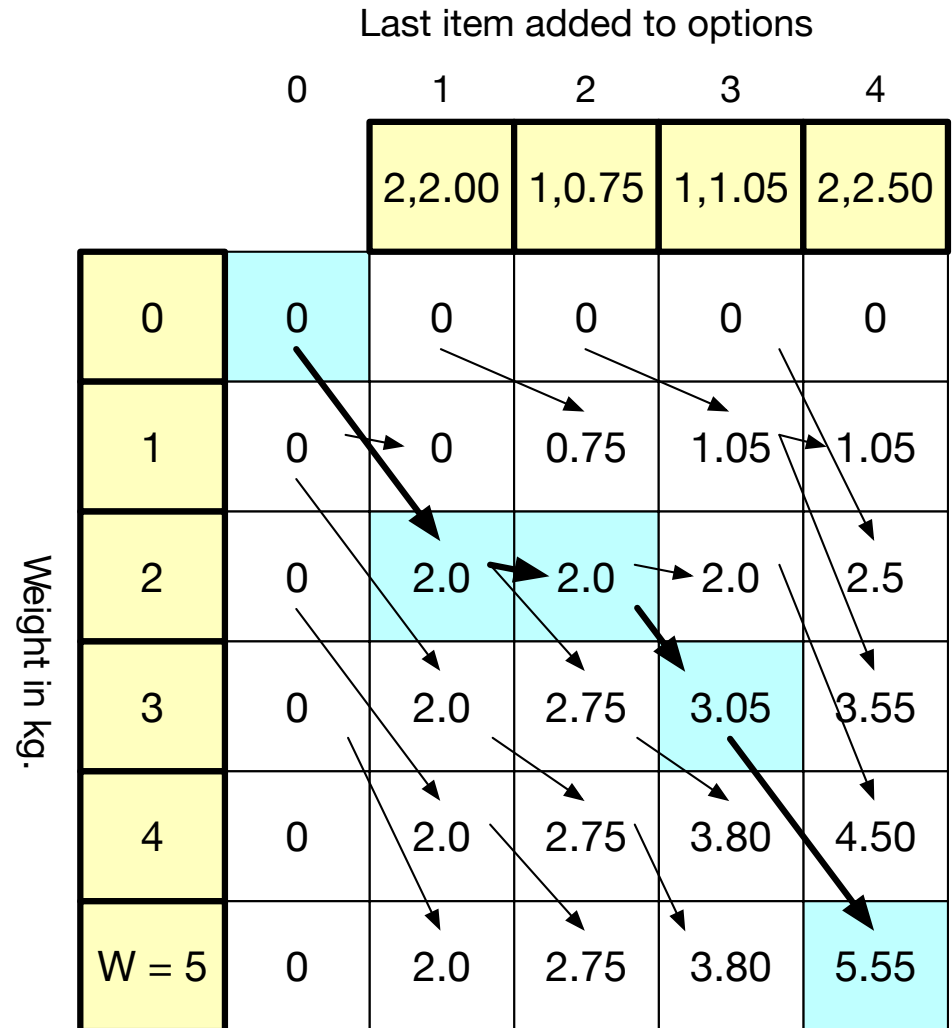
Weight in kg.

Knapsack Pseudocode

```
1 define knapsack( $W : \text{int}, w : \text{int}[1..n], v : \text{float}[1..n]$ ):  
2    $\text{dp} := \text{new array}(0..W, 0..n)$   
3   for  $i \in 0..n$ :  $\text{dp}[0, i] := 0.0$   
4   for  $r \in 0..W$ :  $\text{dp}[r, 0] := 0.0$   
5   for  $r \in 1..W$ :  
6     for  $i \in 1..n$ :  
7       if  $w[i] > r$ :  
8          $\text{dp}[r, i] := \text{dp}[r, i - 1]$   
9       else:  
10         $\text{dp}[r, i] := \max(\text{dp}[r, i - 1], v[i] + \text{dp}[r - w[i], i - 1])$   
11   return  $\text{dp}[W, n]$ 
```

Backtracking

```
1 items := empty list
2  $(r, i) = (W, n)$ 
3 while  $i > 0$ :
4     if  $\text{dp}[r, i] == \text{dp}[r, i - 1]$ :
5          $i := i - 1$ 
6     else:
7         items.append( $i$ )
8          $i := i - 1$ 
9          $r := r - w[i]$ 
10 return "items"
```



Complexity?

Memory: $O(Wn)$

Compute: $O(Wn)$

But only works if weights are *integers*!

What do we do if they are floats?

Weights are floats?

Use branch and bound.

Pick a subset of items 1 to $j < n$.

Use fractional knapsack for upper bound of value that fits in in remaining space from items $j + 1$ to n .

It is NP-Hard.

Questions?

Slides by Aaron Hillegass